

Comma AI Controls Challenge Progress Report

Evan Ewing

Department of Mechanical Engineering
Clemson University
Clemson, United States
enewing@clemson.edu

Miles Turk

Department of Mechanical Engineering
Clemson University
Clemson, United States
turk2@clemson.edu

Abstract—This paper presents a data-driven Koopman model predictive controller (KMPC) design approach for lateral acceleration tracking of an autonomous on-road vehicle subject to disturbances. Following the well-designed controls challenge by Comma AI, the Koopman model is trained on 5,000 to 20,000 data segments to linearize the nonlinear dynamic vehicle model. A model predictive control (MPC) method is then implemented to control the system subject to constraints and a cost function, and its performance and error are compared with those of standard proportional, derivative and integral (PID) control for competition. This paper finds that the KMPC approach outperforms the baseline PID controller across all cost metrics. This paper argues that the KMPC is an optimal approach for nonlinear control, especially when the dynamic equations of a nonlinear system are unknown.

I. INTRODUCTION

Comma AI, a well-known artificial intelligence and control company in the autonomous vehicle industry, has an active, globally available data-driven control challenge. The Comma AI Controls Challenge [1] provides a dataset of car steering measurements from approximately 12,500 hours of driving across hundreds of car models from over 10 brands [2]. The proposed challenge is to develop a new data-driven controller using the provided dataset, apply time-varying steering inputs to their “TinyPhysics” model, and compete in a competition to achieve the lowest total cost/score. This TinyPhysics model acts as the plant model in the simulation. The evaluation costs are related to the lateral acceleration and lateral jerk of the controlled vehicle. Comma AI provides code for their TinyPhysics model and their evaluation software on GitHub [1]. This well-defined challenge serves as the inspiration for the research and contributions of this paper.

A. Data-Driven Modeling

Nonlinear control problems, such as those posed by this challenge, typically require methods beyond model-predictive control (MPC) when no dynamic model is available for the system. Data-driven approaches can handle the challenges of nonlinear control, and a popular method is the finite-dimensional approximation of the Koopman operator. Approximated Koopman operators have been used to generate a variety of linear models for complex systems, including work that examined quadruped leg dynamics [3]. Their method developed a linear switched system model for quadruped motion on deformable terrain. Papers like this tend to use

the extended dynamic mode decomposition (EDMD) method to obtain a finite-dimensional approximation of the Koopman operator [4]. This relies on building a dictionary function Ψ that maps states to a set of observables, which is then used to approximate the Koopman through regression.

B. Model Predictive Control

Model predictive control is an advanced control method that predicts the system state evolution over a finite horizon to determine optimal control actions over time that achieve a desired reference, based on an associated cost function and set of constraints. This leads the controller to solve an optimization problem. One paper focused on the control of tethered unmanned aerial vehicles (UAV)s that used the MPC approach [6]. The tracking they achieved with a prediction horizon of 18 over 0.1-second time steps demonstrated high performance in a simulation under wave disturbances. However, the system model the controller was based on was a nonlinear model derived from the dynamics of the tethered UAV. This results in a computationally expensive nonlinear optimization problem, due to the system’s complexity, likely rendering the controller incapable of real-time control in a physical implementation. This control method could be applied to a linearized system to solve the vehicle control problem using a linear state-space model approximation of the nonlinear dynamic system.

C. Contributions

This paper seeks to compete in the Comma AI Controls Challenge using a Koopman-based MPC (KMPC). The main contributions of this paper are as follows:

- 1) This paper develops a data-driven model for a predictive controller based on the provided dataset using EDMD and an approximated Koopman operator. Thus, a linear approximation of the system is derived.
- 2) This paper evaluates the accuracy of the approximate model at determining the state evolution of the system over time.
- 3) This paper develops a KMPC using a quadratic optimization problem to predict state evolution of the TinyPhysics model, satisfy constraints, and apply optimal steering inputs to the system.
- 4) This paper simulates the KMPC using the provided TinyPhysics model and evaluation code, trained on

different numbers of data segments, and compares its performance to itself and to standard PID.

II. APPROACH AND METHODOLOGY

A. Data-Driven Modeling

The Comma AI model is based on a bicycle dynamic model, but diverges heavily from the traditional formulation. The strongest difference is that this model is a neural network trained to mimic the bicycle model, and specifically includes residuals as part of its network architecture. This makes the model auto-regressive, meaning the state data available to the controller at a given timestep may be insufficient to predict future state evolutions. An augmented state, given by $\tilde{x}_k$ is used to capture some of the system's history and better capture the dynamics by including information from previous states.

$$\tilde{x}_k = \begin{bmatrix} v_k \\ a_k \\ r_k \\ \ell_k \\ \ell_{k-1} \end{bmatrix}$$

Where v_k is the velocity, a_k is the forward acceleration, r_k is the road roll-induced lateral acceleration, ℓ_k is the lateral acceleration, and the $k-1$ terms capture the history of the system's control input and lateral acceleration output.

From this augmented state, the following dictionary was constructed for use with EDMD. This dictionary was designed to capture the system's nonlinear behavior by including several nonlinear terms, specifically the coupling terms and higher-order terms. Note that velocity terms v are normalized in the dictionary by V_{norm} to prevent their domination of the fitted A and B matrices.

$$\psi(x_k, x_{k-1}) = \left\{ \begin{array}{l} \text{bias + states} \\ 1, \frac{v}{V_{\text{norm}}}, a, r, \ell_k \\ \text{memory} \\ \ell_{k-1}, (\ell_k - \ell_{k-1}) \\ \text{coupling terms} \\ \frac{v}{V_{\text{norm}}} \ell_k, r \ell_k, a \ell_k, \frac{v}{V_{\text{norm}}} r \\ \text{nonlinear terms} \\ \ell_k^2, \ell_k^3, \left(\frac{v}{V_{\text{norm}}} \right)^2, r^2 \end{array} \right\}$$

The bias and state terms are standard terms that capture the system's simple linear dynamics. The memory terms capture how previous lateral accelerations influence the future state evolution. Including additional memory terms derived from the other state variables could provide an increase in model accuracy, but would likely lead to overfitting rather than better dynamics. The coupling terms capture the majority of the nonlinear coupling between the states, such as velocity and lateral acceleration with $\frac{v}{V_{\text{norm}}} \ell_k$ and forward acceleration and lateral acceleration with $a \ell_k$. The nonlinear terms capture the higher-order dynamics of the states and the system's

dependence on them, such as the common relationship with the square of velocity, $(\frac{v}{V_{\text{norm}}})^2$.

This dictionary is used to obtain the A and B matrices through linear regression. Trajectory data is passed through the dictionary in paired samples, where sequential time steps are used to obtain $\psi(x_k)$ and $\psi(x_{k+1})$. A and B are then obtained via fitting the sample data with the following equation:

$$\psi(x_{k+1}) \approx A\psi(x_k) + Bu_k \quad (1)$$

This yields a 15x15 A matrix and a 15x1 B matrix. Another matrix, C , is used to map the observables back to the original state space. This matrix satisfies the following equation, and is also shown. This construction is possible because the dictionary includes states, but the constant bias term and other terms meant to capture nonlinearities need to be zeroed out for this mapping.

$$\begin{aligned} x_{k+1} &= C\psi(x_k) \\ C &= \{ 0, I_5, 0 \} \end{aligned} \quad (2)$$

The validity of the Koopman-derived linear model is tested by examining its accuracy in predicting the state evolution over a one-step and a 40-step rollout. This paper determines the short- and long-horizon accuracy of the model.

B. Model Predictive Control

Using the provided Koopman-derived linear state-space model of the system, a quadratic cost-function MPC approach is used to determine the vehicle's control actions. The TinyPhysics model requires car velocity, forward acceleration, lateral acceleration from road roll, current car lateral acceleration, and steer input [1]. Among these inputs, the only control action the MPC needs to optimize and output is the steer input. The remaining inputs to the TinyPhysics model are provided to the controller in real time and can be treated as states and disturbances. Also, the main purpose of the controller is to track a reference lateral acceleration over time based on the states and parameters. This results in the following MPC cost function over the prediction horizon N at each control interval k , that is modeled off the MPC formulation in [6].

$$J_k^* = \min_U \sum_{i=0}^{N-1} Q (\ell_{k+i} - \ell_{k+i,ref})^2 + R_d (u_{k+i} - u_{k+i-1})^2, \quad (3)$$

subject to

$$\begin{aligned} x_{k+i+1} &= f_{\text{Koopman}}(x_{k+i}, u_{k+i}, d_{k+i}), \\ x_{k+i} &\in \mathcal{X}, \\ u_{k+i} &\in \mathcal{U}, \\ x_0 &= x(0), \end{aligned}$$

with

$$U = [u_k \quad u_{k+1} \quad u_{k+2} \quad \dots \quad u_{k+N-1}].$$

$J_k^* \in \mathbb{R}_0^+$ is the cost function at time interval k ,
 $U \in \mathbb{R}^{1 \times N}$ is the optimized control action over the prediction

horizon N ,

$i \in \mathbb{R}^+$ is the prediction horizon step,

Q is the weight of the lateral acceleration error,

R_d is the input rate weight that minimizes jerk,

$\mathcal{X} \in \mathbb{R}^{5 \times 1}$ is the set of state constraints,

$\mathcal{U} \in \mathbb{R}^{1 \times 1}$ is the set of input constraints,

$f_{\text{Koopman}}(x_{k+i}, u_{k+i}, d_{k+i}) \in \mathbb{R}^{15 \times 1}$ is the Koopman derived discretized linear output model.

This cost function over the prediction horizon N is formulated as a quadratic optimization problem. The optimization problem is solved using sequential least squares programming (SLSQP) to determine the optimal control action at each time step. The parameters of this optimization problem are listed in Table I. The steering control input is within $u \in [-2, 2]$ as limited by the challenge and the states are left unconstrained as there are no other listed constraints in the challenge description [1]. The tracking weight and control rate weights after iterative tuning were set to 1.0 and 4.0.

TABLE I
KOOPMAN MPC PARAMETERS

Parameter	Value	Description	Units
p	3	Prediction horizon	steps
Q	1.0	Tracking weight	—
R_d	4.0	Control rate weight	—
$u_{\min}$	-2	Minimum control input	—
$u_{\max}$	2	Maximum control input	—

The flow of information through the control system in simulation is shown in Figure 1. The Comma AI evaluation program is the backbone of the simulation, injecting references and disturbances into the system. It uses the approximated bicycle model to evolve the state when given a control input. The KMPC receives state information from the script and provides control inputs to the system model. The main feedback comes from the system model's updated states, which are returned to the KMPC at every time step.

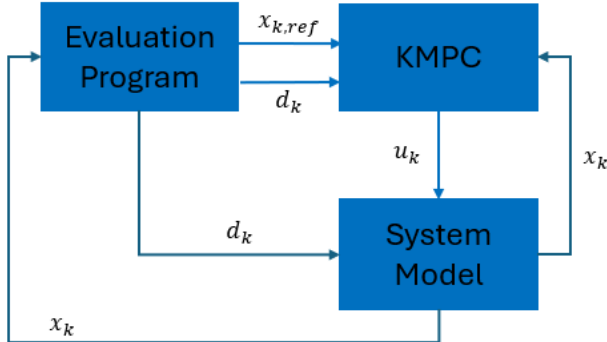


Fig. 1. Diagram of the flow of the control system in simulation.

III. EVALUATION PLAN

The evaluation plan to validate the KMPC is straightforward. The Comma AI Controls Challenge provides its own performance criteria, which are costs related to lateral acceleration ℓ and lateral jerk $\mathcal{J}$ [1]. These cost terms are defined as

$$\ell^* = \frac{\sum(\ell - \ell_{ref})^2}{N} \times 100, \quad (4)$$

$$\mathcal{J}^* = \frac{(\sum(\ell_t - \ell_{t-1}/(\Delta t))^2}{N - 1} \times 100. \quad (5)$$

The evaluation software, eval.py, automatically computes these cost terms and assesses the controller's performance during the simulation by summing the total cost as defined below:

$$C = (\ell^* \times 50) + \mathcal{J}^*.$$

Notice how the lateral acceleration cost is 50 times greater than the lateral jerk cost. Therefore, lateral acceleration tracking was the primary priority. This evaluation method uses the first 5000 trajectories of the dataset and compares two controllers' aggregate costs over the trajectories. For scoring purposes, a custom controller is tested against the PID provided by Comma, which produces an aggregate cost of 111.346 on the given trajectories. The script produces the cost for the PID as well as the custom controller, and if a controller beats the PID baseline, it qualifies for submission for a leaderboard position [1]. This paper's goal was to achieve a score that would appear on the leaderboard if submitted by developing a strong performing KMPC [7].

IV. RESULTS

The Koopman model was able to closely track the reference lateral acceleration when applied to the first 100 control inputs provided alongside the sample trajectories. The model trained on all trajectories achieved a test RMS error for lateral acceleration prediction of 0.0718, and the RMS values for the cross-validation models trained on folds of the trajectories is shown in Table II, and the performance of the model on a sample trajectory is shown in Figure 2.

TABLE II
5-FOLD CROSS-VALIDATION RESULTS (16,000 TRAJECTORIES)

Fold	One-Step RMS	Lat. Accel. RMS
1	0.39872	0.03332
2	0.48592	0.03866
3	2.56263	0.06289
4	1.31242	0.05376
5	0.47109	0.03361
Mean $\pm$ Std	1.04615 $\pm$ 0.82877	0.04445 $\pm$ 0.01184

The controller performance results, evaluated over 5000 trajectories, for both the baseline PID and KMPC are shown in Table III.

The distribution of costs after evaluation over 5000 trajectories for lateral acceleration, lateral jerk, and total cost comparing the baseline PID and KMPC is shown in Figure 3.

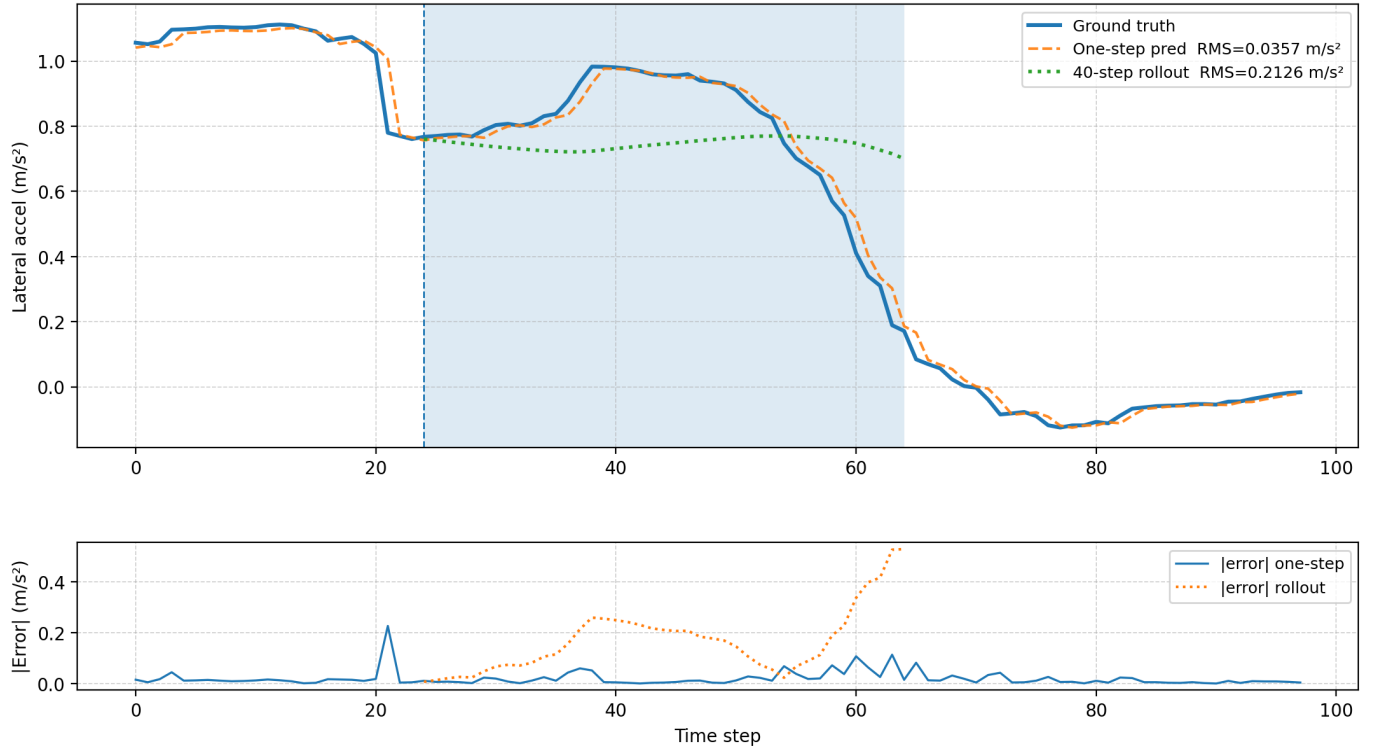


Fig. 2. Plots of trained Koopman model prediction and error on one test segment. The figure shows the model's accuracy for one-step predictions and a 40-step rollout. The prediction error quickly accumulates as the horizon lengthens.

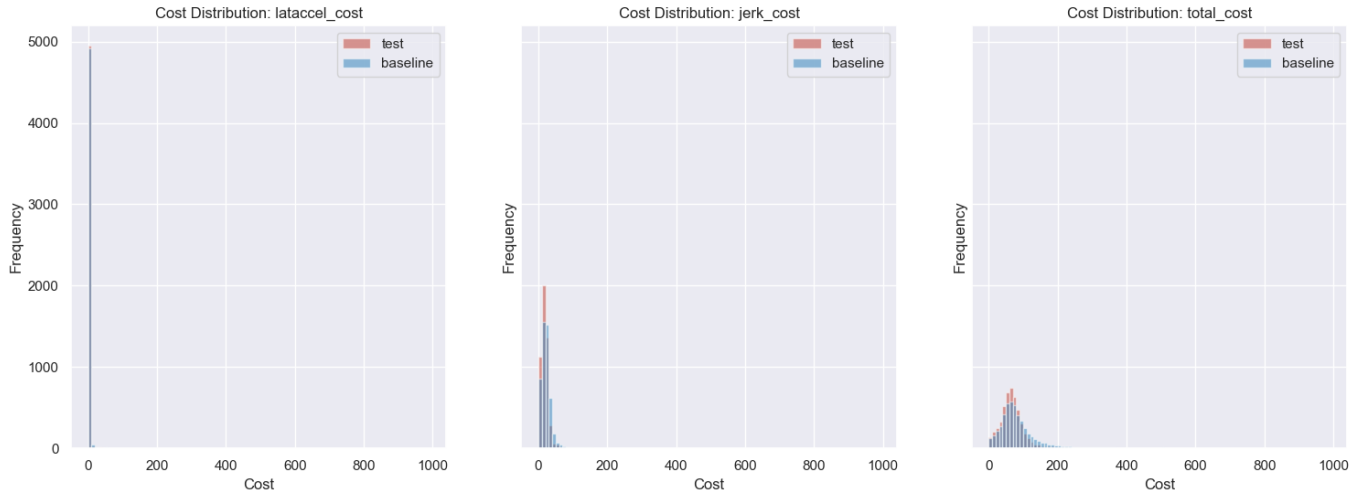


Fig. 3. Plots of the cost distribution for lateral acceleration, lateral jerk, and total for the KMPC trained on 5,000 trajectories and tested on 5,000 trajectories. Note that the distribution of costs for the KMPC trends leftward of the baseline PID controller.

TABLE III
CONTROLLER PERFORMANCE RESULTS

Controller	N_{seg}	ℓ_{cost}	$\mathcal{J}_{\text{cost}}$	Total Cost
Baseline PID	-	1.715	25.607	111.346
Koopman MPC	5000	1.280	20.384	84.392
Koopman MPC	20000	1.283	21.119	85.267

The simulation results for the baseline PID and the KMPC over five sample trajectories, along with the tracking quality, are shown in Figure 4.

V. DISCUSSION

A. Technical Problems and Debugging

This project’s technical problems mostly stem from compatibility with the provided model and evaluation scripts. Comma AI provides a Python script that takes a sample trajectory and a controller as input, and simulates the controller’s response over that trajectory. However, the trajectories use a left-positive steering input convention, while the simulator uses a right-positive convention. The simulation scripts correct for this, but our EDMD fitting was initially performed according to the data sign convention. This led the MPC to output incorrect control actions, as it was effectively operating with a negative B matrix. This was the most significant problem we faced, but it was identified and resolved quickly.

Beyond compatibility issues, the most significant technical problems stemmed from the controller weights. The initial weights induced a tendency toward extreme control actions, leading to lateral acceleration oscillating around the target. This greatly increased the jerk cost and tracking error. Tuning the weights greatly improved performance, enabling the controller to beat the baseline.

B. Koopman Derived Approximate Linear Model

The linear model obtained in this work achieved low RMS error and tracked well when using the provided control inputs, though primarily in one-step predictions. Figure 2 shows that the one-step prediction mainly suffered from a time-lag when compared to the ground truth. The more disappointing result was the 40-step rollout, which saw large error in trials such as these. This model overall performed poorly in regimes with long time horizons, as observed with the MPC controller as well. Earlier iterations of this model produced with smaller dictionaries also had this problem, and usually to a much greater extent. The problem stems mostly from error propagation when extending the lifted state forward during predictions. More accurate A and B can increase accuracy and thus extend the dictionary, providing some benefit. However, dictionary size directly contributes to overfitting, as observed with some larger dictionaries tested. A different approach, such as using a Koopman autoencoder or a bilinear formulation, could overcome these limitations and solve several other dictionary-related problems we observed.

However, there is a deeper problem with any model generated from this data. This dataset lacks a rich enough set

of input excitations and to identify a truly well-fit Koopman model. Most trajectories are somewhat similar and feature mostly smooth lateral acceleration generated by small and correlated inputs. Additionally, features like vehicle velocity remain almost constant over each trajectory and vary little between trajectories. A more comprehensive dataset across a wider range of inputs, speeds, and reference lateral accelerations would yield more valuable information for building a Koopman model like this.

C. KMPC Weights

The tuning of the KMPC weights in Table I does not match closely with the evaluation metrics intuitively. During the tuning process, it was found that when the tracking error for lateral acceleration was weighted more heavily with respect to the control rate weight, the system would saturate the control inputs to drive the lateral acceleration towards the reference, resulting in oscillations around the reference and overall worse trajectory tracking. While this drastically increased the jerk cost, it also resulted in large lateral acceleration error, which is penalized much more harshly within the challenge. Thus, these weights produced non-viable controllers for both the purposes of practical implementation and the controls challenge. Therefore, to smooth the control inputs and eliminate harmonic-like oscillations around the reference lateral acceleration, the control rate weight was set 4 times as heavy as the lateral acceleration error weight. This eliminated oscillatory behavior originating from the MPC.

D. KMPC Performance Versus Baseline PID

The controller performance results in Table III clearly show that both KMPCs outperform the baseline PID in tracking the vehicle’s lateral acceleration and lateral jerk. The cost distribution charts in Figure 3 support this as the costs for the KMPC trained on 5000 trajectories are distributed leftwards of the PID costs. One note here is that the KMPC trained solely on the same 5,000 trajectories used in the evaluation script outperforms the KMPC trained on all 20,000 trajectories. The difference is minimal, and mainly serves to decrease our score for purposes of leaderboard placement, but implementation within a real system would likely use the model trained on all trajectories, as the increased evaluation performance is symptomatic of a slightly overfit model. Compared to the PID baseline, the KMPC produces similar tracking results, as shown in Figure 4. However, the tracking error tended to be lower overall, which significantly reduced the final cost over a given trajectory. The aggregate cost of the KMPC of 84.392 places the controller on the leaderboard in 61st place, validating the controller’s competitive performance as shown in Figure 5. It is likely that this approach could achieve an ever lower cost with some refinement or further parameter tuning.

Furthermore, the simulation results verify KMPC’s ability to control nonlinear systems solely from data. Instead of needing a complex and well-defined dynamic model of a nonlinear system, as in the case of nominal nonlinear MPC, KMPC is capable of controlling the nonlinear model with fast-converging

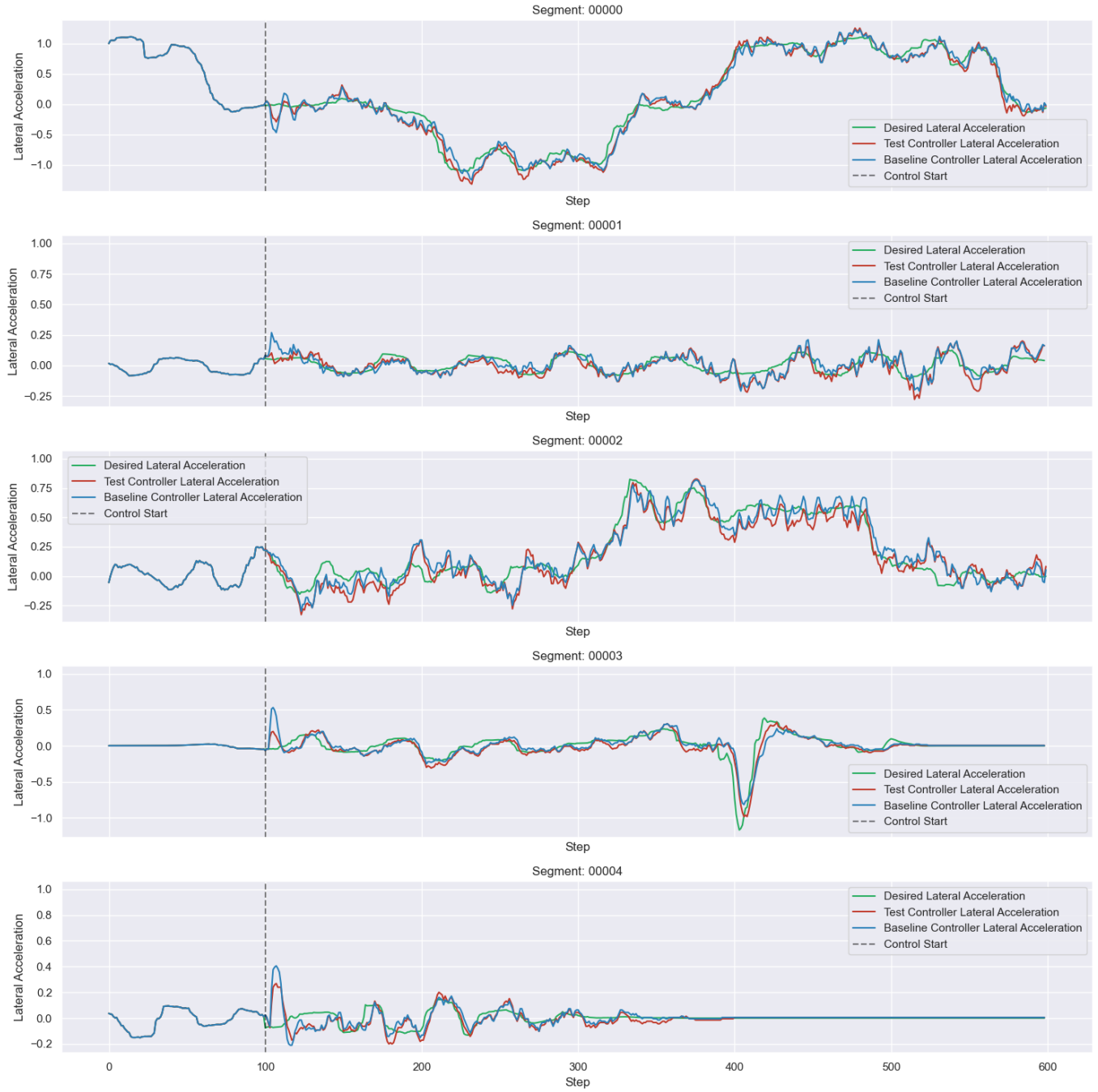


Fig. 4. Simulation graphs of the KMPC trained on 5,000 trajectories for 5 sample trajectories during evaluation. The five subplots show how the controller behaves and tracks the trajectory under various references.

	score	name	method
56	78.868	yiannisha	bayesian optimization
57	79.316	chinmaymirji	PID+TD3
58	79.360	ffmiruz	MPC
59	80.828	donymorph	custom PID
60	83.568	jonoomph8	custom PID
61	88.774	Pranav-Malpore	custom PID

Fig. 5. Current Comma AI controls challenge leader board [7]. The score of 84.392 places the KMPC in 61st place.

linear MPC methods. Aside from the limitation of requiring excitation data from the system rather than dynamic equations, KMPC has clear computational performance advantages over nominal nonlinear MPC, mainly due to its use of a linear model for the system. The primary performance degradation seen with this method stems from the fitted model’s inaccuracies, as these propagate through the KMPC’s prediction horizon. However, these model inaccuracies also exist between simulated and real systems in current MPC implementations based on analytically derived models. Such error propagation is something to be mitigated rather than eliminated entirely, and is not an issue inherent to the controller, but rather the model it employs.

VI. CONCLUSION

This paper develops an EDMD approximation of the Koopman operator for a nonlinear dynamic vehicle system, thereby lifting the system to an approximated linear state-space. After validating the accuracy of the approximated linear model by predicting the state evolution of the system subject to steering control inputs provided with the dataset, a KMPC is derived to calculate the optimal steering control inputs to inject into the system to achieve tracking of lateral acceleration of the vehicle while minimizing lateral torque. The paper found that the KMPC derived from 5,000 trajectories achieved better performance than the KMPC derived from 20,000 trajectories. Based on the evaluation program of the Comma AI controls challenge that the KMPC was designed for, the KMPC also performs significantly better than the baseline PID controller. Therefore, the derived KMPC, in addition to ensuring constraint satisfaction, delivers better performance than a standard nonlinear control method.

A. Limitations

There were several limitations to this research. Since the derived approximate Koopman model is limited in its long-horizon projection accuracy, longer prediction horizons were not viable for the MPC. Typically, MPC converges to better solutions with longer prediction horizons. However, the model estimation error and the injected disturbances into the system led to highly inaccurate projection accuracy. Therefore, the MPC would produce worse control inputs for longer prediction horizons in this particular system.

Another limitation to this research is the lack of ability to control other states of the system. For total dynamic vehicle control, there are three major control inputs: steering, throttle, and braking. In this paper, only applying steering input to the system to control lateral acceleration was possible due to the nature of the challenge and data. A more in-depth exploration of driving control would require a vastly different data-set, likely centered on path-planning and reference positions rather than a reference lateral acceleration.

B. Future Research

This paper leaves ample for expansion on the topic. While the KMPC performed better than the baseline PID, it’s performance still has room for improvement. Possible improvements include tweaking the dictionary that the learned model is based on to capture more of the system dynamic behavior, and layering the KMPC control structure on top of another form of control system such as feed-forward PID. This would allow the KMPC to start from a soft-start control input and optimize it further. This may also improve the controller’s prediction accuracy.

VII. ACKNOWLEDGMENTS

This work was created as a final project for the ME 8930 Data-Driven Control class at Clemson University. The Python programming language used by the controller was supported by Claude AI.

VIII. GITHUB ACCESS

All code written in Python for this paper is publicly available via GitHub at [8].

REFERENCES

- [1] comma.ai, “comma Controls Challenge v2,” GitHub, 2025. https://github.com/commaai/controls_challenge (accessed Feb. 27, 2026).
- [2] comma.ai, “commaSteeringControl,” Github, 2025. <https://github.com/commaai/comma-steering-control> (accessed Feb 27, 2026).
- [3] K. Krolicki et al., “Modeling Quadraped Leg Dynamics on Deformable Terrains using Data-driven Koopman Operators,” *IFAC-PapersOnLine*, vol. 55, no. 37, pp. 420–425, 2022, 2nd Modeling, Estimation and Control Conference (MECC 2022). <https://doi.org/10.1016/j.ifacol.2022.11.219> (accessed Mar. 23, 2026).
- [4] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, ‘A Data-Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition’, *Journal of Nonlinear Science*, vol. 25, no. 6, pp. 1307–1346, June 2015. <https://doi.org/10.48550/arXiv.1408.4408>
- [5] S. S. K. S. Narayanan, D. Tellez-Castro, S. Sutavani, and U. Vaidya, “SE(3) Koopman-MPC: Data-driven Learning and Control of Quadrotor UAVs,” *IFAC-PapersOnLine*, vol. 56, no. 3, pp. 607–612, 2023, 3rd Modeling, Estimation and Control Conference (MECC 2023). <https://doi.org/10.1016/j.ifacol.2023.12.091> (accessed Mar. 23, 2026).

- [6] V. Prkačin, I. Palunko, and I. Petrović, "Trajectory tracking and tether tension control of tethered aerial vehicle using Nonlinear Model Predictive Control," **Control Engineering Practice**, vol. 162, p. 106346, 2025. <https://www.sciencedirect.com/science/article/pii/S0967066125001091> (accessed Mar. 21, 2026).
- [7] comma.ai, "comma.ai leaderboard," 2026. <https://comma.ai/leaderboard> (accessed Apr. 13, 2026).
- [8] MaxMilo3, "controls_challenge," 2026. https://github.com/MaxMilo3/controls_challenge (accessed May. 1, 2026).